

INFORME TÉCNICO: ARTEFACTOS DE DISEÑO DE SOFTWARE BAJO EL PARADIGMA ORIENTADO A OBJETOS Y ARQUITECTURA MVC

**SISTEMA DE GESTIÓN LOGÍSTICA PARA TRANSPORTE DE ENCOMIENDAS
(EIEN-SYSTEM V1.2)**

APRENDIZ: David Alejandro Meyer Romero

SERVICIO NACIONAL DE APRENDIZAJE (SENA)
TECNÓLOGO EN ANÁLISIS Y DESARROLLO DE SOFTWARE (ADSO)
FASE DE PLANEACIÓN – ACTIVIDAD GA4-220501095-AA2
BOGOTÁ D.C.

2026

INTRODUCCIÓN

El presente informe técnico detalla el diseño de la arquitectura de software para el Sistema de Gestión Logística de Transporte de Encomiendas (**Eien-System V1.2**), aplicando los principios fundamentales del paradigma de Programación Orientada a Objetos (POO) y estructurando sus componentes bajo el patrón arquitectónico **Modelo-Vista-Controlador (MVC)**.

En el desarrollo de sistemas de software de alta disponibilidad y concurrencia, como lo requiere el sector logístico, la mezcla indiscriminada de la lógica de presentación con las reglas de negocio y el acceso a datos genera código acoplado, rígido y propenso a fallas críticas.

A través de la implementación de patrones de diseño avanzados como *Singleton* y *Repository*, este documento justifica una propuesta arquitectónica orientada a la mantenibilidad, escalabilidad y modularidad, asegurando una transición limpia hacia la fase de codificación en entornos empresariales.

OBJETIVOS

1. Objetivo General

Diseñar e implementar el modelo estructural avanzado y la arquitectura de software para **Eien-System V1.2** utilizando el Lenguaje de Modelado Unificado (UML), incorporando buenas prácticas de diseño orientado a objetos y el patrón MVC para garantizar una estructura desacoplada y escalable.

2. Objetivos Específicos

- Organizar el sistema en paquetes lógicos diferenciados (view, controller, model) para delimitar estrictamente las responsabilidades de cada componente del software.
- Aplicar el patrón creacional *Singleton* en la clase de utilidad de conexión para optimizar y centralizar de manera segura el acceso a los recursos de la base de datos relacional.
- Mitigar la duplicidad de código en la capa de persistencia mediante la refactorización e introducción de la clase abstracta genérica `AbstractRepository<T>`.
- Modelar e integrar la nueva entidad de negocio `ProveedorLogistico` dentro del Modelo de Dominio, justificando su relación de agregación con su respectiva capa de persistencia.

JUSTIFICACIÓN ARQUITECTÓNICA MODELO-VISTA-CONTROLADOR (MVC)

La implementación del patrón MVC en **Eien-System V1.2** responde a la necesidad de construir una solución de software empresarial desacoplada.

La separación de responsabilidades (*Separation of Concerns*) se ejecuta distribuyendo el sistema en tres capas lógicas estrictas:

- **Capa Vista (view)**
 - Compuesta por interfaces gráficas de usuario (FrmLogin, FrmDespacho).
 - Su única responsabilidad es capturar los datos de entrada del operario y presentar de forma clara los resultados del procesamiento, sin contener jamás lógica de negocio ni sentencias SQL.
- **Capa Controlador (controller)**
 - Encarnada por LogisticaController, actúa como el cerebro intermedio del sistema.
 - Recibe los estímulos lógicos de la vista, coordina las reglas de validación y delega las tareas pesadas a la capa de persistencia.
- **Capa Modelo (model)**
 - Contiene la lógica del negocio pura (entity), los contratos de acceso a datos (repository) y la infraestructura de conectividad (util).
 - El modelo desconoce por completo qué interfaz visual se está usando, lo que permite cambiar el frontend en el futuro sin alterar una sola línea de lógica interna.

TABLA RESUMEN: REQUERIMIENTOS ↔ ENTIDADES ↔ ACCIONES

Para garantizar la trazabilidad entre las necesidades del negocio de transportes y el diseño de artefactos UML, se presenta la matriz de correspondencia técnica:

Tipo	Requerimiento Funcional	Entidades Principales	Controlador Responsable	Métodos / Operaciones Implementadas
RF01	Autenticación y Control de Acceso Operativo	Usuario, Persona	LogisticaController	procesarAutenticacion(usuario, clave)
RF02	Admisión y Registro Digital de Encomiendas	Paquete, PaqueteLocal, PaqueteNacional	LogisticaController	registrarEnvio(codigo, peso, destino)
RF03	Gestión y Control de Proveedores Logísticos	ProveedorLogistico	LogisticaController	registrarNuevoProveedor(nombre, contacto)
RNF03	Seguridad y Aislamiento de Capas de Datos	Todas las entidades	AbstractRepository<T>	getConnection(), createEntity()

ESPECIFICACIÓN TÉCNICA DE PATRONES DE DISEÑO EN LA CAPA MODELO

La arquitectura de persistencia de **Eien-System V1.2** ha sido optimizada para evitar el desperdicio de recursos de hardware y eliminar la redundancia de código en el backend.

1. Patrón Creacional: Singleton (Conexion)

La clase de utilidad Conexion (ubicada en el paquete util) restringe la instanciación de su propio objeto mediante un **constructor estrictamente privado** (- Conexion()).

- **Justificación**

- Las conexiones a bases de datos relacionales (como MySQL o PostgreSQL) son recursos críticos y costosos en memoria.
- El uso de Singleton garantiza que exista una **única instancia del gestor de conexiones** en toda la aplicación a través del método de acceso global + getInstance().

- **Seguridad Concurrentes**

- Se implementa conceptualmente el mecanismo de *Double-Checked Locking* combinado con la propiedad de visibilidad volatile en su atributo - instance, garantizando que la inicialización perezosa (*lazy initialization*) sea completamente segura frente a hilos concurrentes (*thread-safe*).

2. Patrón Estructural: Repository Refactorizado (AbstractRepository<T>)

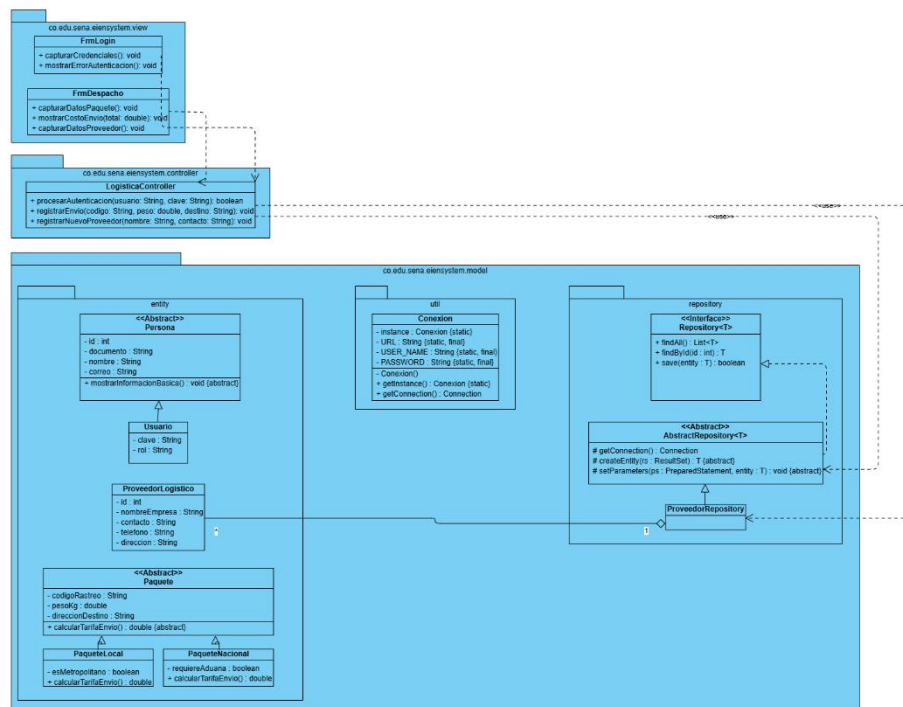
Para dar cumplimiento a la directriz técnica de optimización y evitar la duplicidad de métodos repetitivos (como la gestión manual de conexiones en cada tabla), se implementó una estructura de herencia avanzada en el paquete repository:

- **La Interfaz Genérica Repository<T>**
 - Define el contrato formal de operaciones CRUD básicas (findAll(), findById(), save()) abstrayendo por completo la tecnología de persistencia de la lógica de negocio.
- **La Clase Base Abstracta AbstractRepository<T>**
 - Actúa como la superclase que centraliza la lógica redundante.
 - Absorbe el método # getConnection() con modificador de acceso protegido (protected), permitiendo que cualquier repositorio hijo (como ProveedorRepository o UsuarioRepository) herede de forma nativa la conexión sin necesidad de reescribirla.
- **Mapeo Polimórfico**
 - Declara los métodos abstractos # createEntity(rs: ResultSet) y # setParameters(ps: PreparedStatement) obligando a las clases hijas a implementar únicamente la conversión de datos que sea específica para su respectiva tabla.

ARTEFACTO UML: DIAGRAMA DE CLASES ARQUITECTÓNICO (MVC)

A continuación, se integra el modelo estructural de **Eien-System V1.2**.

En este artefacto se evidencia claramente la división por paquetes organizacionales (view, controller, model), la implementación del patrón Singleton en la conectividad, la clase abstracta de persistencia base y la incorporación de la nueva entidad ProveedorLogistico con su correspondiente repositorio mediante una relación de agregación con multiplicidad 1 a *.



URL Imagen: [Diagrama de dominio v 1.2.png](#)

CONCLUSIONES

- La separación del sistema en las capas de **Modelo, Vista y Controlador (MVC)** reduce drásticamente el acoplamiento del software.

Esto garantiza que modificaciones futuras en la interfaz de usuario (como migrar de una aplicación de escritorio a una interfaz web basada en HTML, CSS y JavaScript) no afecten en absoluto las reglas de negocio ni la estructura de datos interna del sistema logístico.

- El uso del patrón **Singleton** en la gestión de conectividad demuestra ser una práctica indispensable para el control de recursos críticos en el servidor, impidiendo la apertura desmesurada de conexiones concurrentes a la base de datos y optimizando el rendimiento general del entorno de ejecución.
- La refactorización de los componentes de acceso a datos mediante la creación de la superclase **AbstractRepository<T>** soluciona de forma definitiva el problema de duplicidad de código en el backend.

Al centralizar la lógica común de JDBC y heredarla a través de métodos protegidos (#), el diseño de software se vuelve altamente extensible, permitiendo la adición de nuevas entidades de negocio en cuestión de minutos y elevando los estándares de mantenibilidad de **Eien-System**.